

DarkSage

Database Design

Version 0.2.1 — Draft

Owner: TheSinnerMan

Classification: Internal

Wisdom Over Noise

Source repository: [LightSwornPVP/DarkSage](#)

Source baseline commit: [34a65818d48f596122281075e18c6e5f36ec93b5](#)

Generation date: 2026-07-25T13:31:10-04:00

Document ID: DS-005

Document Control

Document ID	DS-005
Title	Database Design
Version	0.2.1
Status	Draft
Owner	TheSinnerMan
Classification	Internal
Repository	LightSwornPVP/DarkSage
Created	2026-07-24
Last Updated	2026-07-24

Status lifecycle: Draft → Under Review → Approved → Superseded/Deprecated.

Revision History

Version	Date	Author	Summary
0.1.0	2026-07-24	TheSinnerMan	First controlled draft. Defines entity/schema design for the shared models ARCHITECTURE.md §6 names, on the SQLite-first strategy DS-ARC-016 already establishes, cross-referenced to DS-002's requirement families.
0.2.0	2026-07-24	TheSinnerMan	Repair pass addressing independent audit findings DS-005-A01 through A05. A01: added Transaction entity (DS-DB-025), Position and DS-DB-023 updated to reference it. A02: added Alert (DS-DB-026) and Notification (DS-DB-027) entities as new §13, sections renumbered 13→14 through 18→19 accordingly. A03: reclassified StrategyProfile and TradeDecision to Committed/MVP (Phase-1 base model, per ROADMAP.md/DS-ARC-005), reclassified BrokerState Future/Exploratory → Planned, made ScanConfiguration.profile_id nullable so it is independently satisfiable without the Planned UserProfile feature. A04: corrected Candle's foreign-key target (DS-DB-013 → DS-DB-014, SecurityIdentity); added signal_id to Signal and regime_id to MarketRegime as their primary identifiers, matching the foreign keys that already referenced them. A05: split StrategyProfile and TradeDecision into a Committed Phase-1 core and a Planned later-phase extension (Phase 2 for StrategyProfile's configuration/risk fields; Phase 6/7 for TradeDecision's AI-provider/pipeline fields), so neither Committed base model requires a Planned capability to be valid.
0.2.1	2026-07-24	TheSinnerMan	Consolidated cleanup pass: corrected DS-DB-004's stale Alert reference (Alert was already added as DS-DB-026 in the 0.2.0 A02 repair; the Key Relationships text still said "not yet in this document's entity set") to cite DS-DB-026 directly. Corrected DS-DB-027's channel field, which mislabeled DS-ALT-002 (in-app notification delivery) as "Committed" although DS-ALT-002 is formally Planned — both in-app (DS-ALT-002) and external (DS-ALT-003) channels are now correctly stated as Planned. No entity, classification, or relationship was otherwise changed.

Table of Contents

Document Control	2
Revision History	3
Table of Contents	4
1. Purpose	6
2. Scope	6
3. Audience	6
4. Definitions	6
5. Database Strategy Recap	6
6. Core Market Data Entities	6
DS-DB-001 — Candle	6
DS-DB-002 — Quote	7
DS-DB-003 — MarketRegime	7
7. Signal, Strategy, and Backtesting Entities	7
DS-DB-004 — Signal	8
DS-DB-005 — StrategyProfile	8
DS-DB-006 — BacktestResult	9
DS-DB-007 — TradeDecision	9
8. Portfolio and Execution Entities	10
DS-DB-008 — Portfolio	10
DS-DB-009 — Position	10
DS-DB-025 — Transaction	10
DS-DB-010 — Order	11
DS-DB-011 — RiskState	11
DS-DB-012 — BrokerState	12
9. Chart Entities	12
DS-DB-013 — ChartAnnotation	12
10. Product and Platform Entities	13
DS-DB-014 — SecurityIdentity	13
DS-DB-015 — UserProfile	13
DS-DB-016 — Preferences	13
DS-DB-017 — WorkspaceLayout	13
DS-DB-018 — Watchlist / WatchlistItem	14

11. Security and Operations Entities	14
DS-DB-019 — IntegrationCredential	14
DS-DB-020 — AuditLogEntry	14
12. Scanner Entities	15
DS-DB-021 — ScanConfiguration	15
DS-DB-022 — ScanResult	15
13. Alerts and Notifications	15
DS-DB-026 — Alert	16
DS-DB-027 — Notification	16
14. Cross-Cutting Data Integrity Requirements	17
DS-DB-023 — Append-Only Audit and Transaction Tables	17
DS-DB-024 — Migration Discipline	17
15. Non-Goals	17
16. Dependencies	17
17. Risks and Constraints	17
18. Verification Approach	18
19. References	18
Appendix A — Open Questions	18

1. Purpose

DS-005 defines DarkSage's data entities: their purpose, key fields, relationships, and integrity constraints. It builds directly on ARCHITECTURE.md §6 (Shared Models) and DS-ARC-005/DS-ARC-016 (which establish *that* shared models and SQLite-first storage exist); DS-005 defines *what those models actually contain*. This is a schema-design document, not a full DDL implementation artifact — exact column types, indexes, and migration tooling are left to implementation, while entity shape, required fields, relationships, and invariants are normative here.

2. Scope

Covers entity design for: core market data (Candle, Quote, MarketRegime), signals and strategy (Signal, StrategyProfile, TradeDecision, BacktestResult), portfolio and execution (Position, Transaction, Portfolio, Order, BrokerState, RiskState), charting (ChartAnnotation), product/platform state (UserProfile, Preferences, Watchlist, WorkspaceLayout, SecurityIdentity), security/operations (IntegrationCredential, AuditLogEntry), scanning (ScanConfiguration, ScanResult), and alerts/notifications (Alert, Notification).

Does not cover: exact SQL DDL/migration tooling (implementation detail), API request/response shapes (DS-006), or UI data-binding (DS-007).

3. Audience

Backend/database contributors, independent auditors.

4. Definitions

See DS-001 §24, DS-002 §4, DS-004 §4.

5. Database Strategy Recap

Per DS-ARC-016: SQLite is the initial database; PostgreSQL/TimescaleDB migration is a later, not-yet-justified decision (Phase 12). All entity designs in this document assume SQLite's type system (dynamic typing, no native array/JSON column enforcement beyond TEXT-stored JSON) as the baseline implementation target, without precluding a future migration.

6. Core Market Data Entities

DS-DB-001 — Candle

Release Classification: Committed / MVP | **Governing Source:** ARCHITECTURE.md §6; ROADMAP.md Phase 1 ("Candle model"); DS-MKT-001, DS-MKT-002

Purpose: Store OHLCV market data at a given symbol/timeframe/timestamp — the base unit historical and real-time price analysis builds on.

Key Fields: symbol_id (FK → SecurityIdentity, DS-DB-014), timeframe, timestamp (UTC), open, high, low, close, volume, source_provider, ingested_at, is_adjusted (bool, corporate-action adjustment flag).

Key Relationships: Many Candles per SecurityIdentity; consumed by DS-ARC-007 (Indicator Engine), DS-ARC-008 (Chart Adapter), DS-ARC-010 (Backtest Engine).

Constraints/Invariants: Unique on (symbol_id, timeframe, timestamp); immutable once stored except through an explicit, logged correction (DS-MKT-002); is_adjusted distinguishes split/dividend-adjusted series from raw, consistent with DS-MKT-006's corporate-action handling once built.

Testing: Uniqueness-constraint test; correction-audit-trail test.

DS-DB-002 — Quote

Release Classification: Committed / MVP | **Governing Source:** ARCHITECTURE.md §6; ROADMAP.md Phase 1; DS-MKT-003, DS-PRD-008

Purpose: Store the current/last-known price point for a symbol, with the state metadata DS-PRD-008 requires.

Key Fields: symbol_id (FK), price, bid, ask, size, timestamp, data_state (enum: current/delayed/stale/historical/simulated), delay_seconds (nullable), source_provider.

Key Relationships: One current Quote per SecurityIdentity per provider; referenced by Position (DS-DB-009) for mark-to-market.

Constraints/Invariants: data_state and delay_seconds are never null when data_state != current; a Quote record is a snapshot, not mutated in place — a new record supersedes rather than overwrites (supports DS-PRD-008's auditability of "what changed").

Testing: State-transition test (current → delayed → stale) per DS-MKT-004's threshold logic.

DS-DB-003 — MarketRegime

Release Classification: Planned | **Governing Source:** ARCHITECTURE.md §6; PROJECT_SPEC.md §13; ROADMAP.md Phase 3

Purpose: Store classified market-environment state (bull/bear/sideways, volatility regime, etc.) used to segment strategy performance (PROJECT_SPEC.md §7/§13).

Key Fields: regime_id (primary identifier), regime_type, effective_date, confidence, classification_method_version.

Key Relationships: Referenced by BacktestResult (DS-DB-006) and StrategyProfile performance segmentation for regime-based analysis.

Constraints/Invariants: classification_method_version is recorded so a later change to the classification methodology doesn't silently reinterpret historical regime labels.

Testing: Not yet applicable at Planned classification; deferred to Phase 3 authoring.

7. Signal, Strategy, and Backtesting Entities

DS-DB-004 — Signal

Release Classification: Committed / MVP | **Governing Source:** ARCHITECTURE.md §6; ROADMAP.md Phase 1 ("Signal model"); PROJECT_SPEC.md §16; DS-SCN-002/003

Purpose: Represent a single scan/strategy output: a candidate opportunity with its supporting evidence and score.

Key Fields: signal_id (primary identifier), symbol_id (FK), strategy_id (FK, nullable for pure-scanner signals), direction, entry/stop/targets (nullable until a strategy assigns them), confidence, quantitative_score, technical_score, fundamental_score, sentiment_score, detected_patterns (JSON), indicators_snapshot (JSON), reasoning (text), grade (A+/A/B/C/D per PROJECT_SPEC.md §16), generated_at, expires_at (nullable).

Key Relationships: Many Signals per SecurityIdentity; optionally linked to a StrategyProfile (DS-DB-005); optionally referenced as an Alert (DS-DB-026, Planned) condition target via condition_definition, per DS-DB-026's own Key Relationships.

Constraints/Invariants: grade is derived from measurable inputs (TRADING_RULES.md "Signal Grades"), never assigned by unstructured AI judgment alone (DS-PRD-004 analog for scoring); a rejected signal retains its rejection reason (PROJECT_SPEC.md §17, Why-Trade/Why-Not-Trade), not deleted.

Testing: Grade-derivation determinism test; rejection-reason completeness test.

DS-DB-005 — StrategyProfile

Release Classification: Committed / MVP (Phase-1 base model only — see field split below; full strategy-construction functionality remains Planned) | **Governing Source (corrected in the DS-005-A03 repair):** ROADMAP.md Phase 1 ("StrategyProfile model") places the base model in the Committed Phase-1 subset; ARCHITECTURE.md §10; DS-ARC-005 (Committed, narrowed Phase-1 model list).

Purpose: Represent a versioned, auditable strategy definition — as a Phase-1 base record shape now (referenced by Signal's optional strategy_id), with full strategy-construction richness arriving in Phase 2.

Key Fields (Phase-1 core, Committed): strategy_id (primary identifier), name, version, status (Experimental/Watch/Active/Reduced/Suspended), created_at, superseded_by (nullable, FK → StrategyProfile, self-referential version chain).

Key Fields (Phase-2 extension, Planned — nullable/absent until DS-STR-001 exists): supported_timeframes (JSON list), supported_instruments (JSON list), configuration (JSON), risk_assumptions (JSON).

Key Relationships: One-to-many with Signal (DS-DB-004), TradeDecision (DS-DB-007), BacktestResult (DS-DB-006).

Constraints/Invariants: A logic change creates a new version row (superseded_by chain), never an in-place mutation of a prior version's configuration (AGENTS.md "No Silent Strategy Changes"); this invariant applies once the Phase-2 configuration field is populated and is not required for the Phase-1 core to be valid on its own.

Testing: Phase-1 core: identity/versioning-chain integrity test. Phase-2 extension: immutability-of-prior-version configuration test (once DS-STR-001 exists).

DS-DB-006 — BacktestResult

Release Classification: Planned | **Governing Source:** ARCHITECTURE.md §6, §13; ROADMAP.md Phase 2; DS-BKT-001/002/003/004

Purpose: Store a reproducible backtest run's configuration and output.

Key Fields: backtest_id, strategy_id (FK, version-pinned), date_range_start/end, cost_assumptions (JSON — DS-BKT-002), data_snapshot_reference, results (JSON: trade_count, win_rate, expectancy, profit_factor, max_drawdown, sharpe, sortino, sample_size_confidence), regime_id (FK, nullable), executed_at.

Key Relationships: Many-to-one with StrategyProfile (version-pinned, not just strategy_id, so a later strategy edit doesn't retroactively reinterpret a past result); many-to-one with MarketRegime for regime-segmented analysis.

Constraints/Invariants: Immutable once written (DS-BKT-001's reproducibility requirement); strategy_id pins the exact version used, not a mutable reference to "current" strategy state.

Testing: Reproducibility regression test (same strategy version + date range + cost assumptions → identical results).

DS-DB-007 — TradeDecision

Release Classification: Committed / MVP (Phase-1 core only — see field split below; pipeline/AI extension fields remain Planned) | **Governing Source (corrected in the DS-005-A03/A05 repair):** ROADMAP.md Phase 1 ("TradeDecision model") places the base model in the Committed Phase-1 subset; DS-ARC-005 (Committed, narrowed Phase-1 model list). Full pipeline integration is ARCHITECTURE.md §14 / ROADMAP.md Phase 7 (DS-ARC-011, Planned); AI provider involvement is Phase 6 (DS-ARC-013, Planned).

Purpose: Represent a Trade Proposal at a Phase-1-safe base level — an idea/candidate a Signal or Strategy could act on — without requiring the Phase-6 AI provider layer or the Phase-7 TradeValidationPipeline to exist for the record to be valid. Once those capabilities exist, the same table extends to become the full pipeline audit record (DS-OPS-002's backbone).

Key Fields (Phase-1 core, Committed): decision_id (primary identifier), signal_id (FK), strategy_id (FK, nullable), proposed_size, proposed_direction, final_disposition (Phase-1-safe enum: proposed / rejected only — validated/executed are added as valid values once the Phase-7 pipeline exists, see extension below), created_at.

Key Fields (Phase-6/Phase-7 extension, Planned — nullable/absent until those capabilities exist): pipeline_stage_results (JSON — one entry per TradeValidationPipeline stage, populated only once DS-ARC-011 is implemented), ai_provider_used (nullable, populated only once an AI provider, DS-ARC-013, exists), ai_output_raw (nullable, text, for audit, same gating). final_disposition gains validated/executed as valid values only once the Phase-7 pipeline exists.

Key Relationships: One-to-one with an eventual Order (DS-DB-010) if final_disposition = executed (Phase 7+); many-to-one with Signal and StrategyProfile.

Constraints/Invariants: The Phase-1 core (decision_id, signal_id, strategy_id, proposed_size/direction, final_disposition ∈ {proposed, rejected}, created_at) is independently valid and complete without any Phase-6/7 field populated. Once populated, pipeline_stage_results records every stage's outcome even on early rejection — a rejected

proposal still shows which stage rejected it and why (TRADING_RULES.md Why-Trade/Why-Not-Trade); this table is the audit-trail backbone for DS-OPS-002 once that extension is in use.

Testing: Phase-1 core: record-validity test with no extension fields populated. Phase-6/7 extension (once applicable): pipeline-stage-completeness audit test; audit-reconstruction test (DS-OPS-002).

8. Portfolio and Execution Entities

DS-DB-008 — Portfolio

Release Classification: Planned | **Governing Source:** ARCHITECTURE.md §6, §19; ROADMAP.md Phase 4; DS-PRT-004

Purpose: Represent a named collection of positions (a paper account, or in future a live account) with aggregate metrics.

Key Fields: portfolio_id, name, account_type (paper/live), cash_balance, created_at.

Key Relationships: One-to-many with Position (DS-DB-009); referenced by RiskState (DS-DB-011) for portfolio-level risk budget.

Constraints/Invariants: account_type = live requires the Stage 4 / Phase 13 prerequisites (DS-ARC-018) to be satisfied before any Order (DS-DB-010) may reference it with a non-paper Broker Adapter.

Testing: Not yet applicable at Planned classification.

DS-DB-009 — Position

Release Classification: Planned | **Governing Source:** ARCHITECTURE.md §6; ROADMAP.md Phase 2/4/7; DS-PRT-001

Purpose: Represent a held quantity of a security within a Portfolio, derived from Transaction history.

Key Fields: position_id, portfolio_id (FK), symbol_id (FK), quantity, cost_basis, opened_at, closed_at (nullable).

Key Relationships: Derived from the append-only Transaction ledger (DS-DB-025) — a Position is computed by aggregating a symbol's Transactions within a Portfolio, not stored as independently-entered data.

Constraints/Invariants: Quantity/cost_basis are computed deterministically from the transaction log (DS-PRD-004); a Position reduced to zero is closed (closed_at set), not deleted.

Testing: Position-derivation regression test against fixture transaction sequences.

DS-DB-025 — Transaction

Release Classification: Planned | **Governing Source (added in the DS-005-A01 repair):** DS-PRT-002 (Transaction Recording, Planned) — requires immutable transaction recording that DS-PRT-001/003 derive positions and performance from; DS-ARC-023 (Portfolio Architecture, Planned) requires a Transaction/Position data model; DS-DB-023 (Committed, Append-Only Audit and Transaction Tables) already anticipates this entity.

Purpose: The authoritative, append-only transaction ledger Position (DS-DB-009) derives from, and the source of truth for realized/unrealized performance (DS-PRT-003).

Key Fields: `transaction_id` (primary identifier), `portfolio_id` (FK → Portfolio, DS-DB-008), `symbol_id` (FK → SecurityIdentity, DS-DB-014), `transaction_type` (buy/sell/dividend/split-adjustment/fee/other), `quantity`, `price`, `fee` (nullable), `executed_at`, `recorded_at`, `source` (manual/import — DS-DAT-002), `reverses_transaction_id` (nullable, self-referential FK → Transaction, for corrections).

Key Relationships: Many-to-one with Portfolio and SecurityIdentity; Position (DS-DB-009) is computed deterministically by aggregating a symbol's Transactions within a Portfolio; DS-PRT-003's realized/unrealized performance is likewise derived from this ledger, not stored independently.

Constraints/Invariants: Append-only (DS-DB-023) — a confirmed Transaction is never updated or deleted; corrections use an explicit `reverses_transaction_id`-linked adjusting entry (DS-PRT-002's "explicit, logged reversing/adjusting entry, never silent edits"). `quantity/price/fee` are the deterministic inputs DS-PRD-004 requires Position/performance calculations to reproduce exactly given the same transaction set.

Testing: Append-only/no-update-or-delete enforcement test; reversal-chain integrity test; position-derivation determinism test against fixture transaction sequences (shared with DS-DB-009's own test).

DS-DB-010 — Order

Release Classification: Planned | **Governing Source:** ARCHITECTURE.md §6, §14; ROADMAP.md Phase 7; DS-ARC-011

Purpose: Represent an order submitted to a Broker Adapter (paper initially) after passing the full TradeValidationPipeline.

Key Fields: `order_id` (unique, idempotency key per TRADING_RULES.md "Duplicate Order Protection"), `trade_decision_id` (FK), `broker_environment` (paper/live), `symbol_id`, `quantity`, `order_type`, `status` (pending/filled/partial/cancelled/rejected), `submitted_at`, `filled_at` (nullable), `fill_price` (nullable).

Key Relationships: One-to-one with the TradeDecision that produced it; many-to-one with BrokerState (DS-DB-012) for reconciliation.

Constraints/Invariants: `order_id` is globally unique and idempotent — a retry must not create a duplicate order (TRADING_RULES.md); `broker_environment = live` is disallowed until Stage 4/Phase 13 prerequisites are met (same gate as DS-DB-008).

Testing: Idempotency/duplicate-submission test (ROADMAP.md Phase 7 exit criterion).

DS-DB-011 — RiskState

Release Classification: Planned | **Governing Source:** ARCHITECTURE.md §6, §17; ROADMAP.md Phase 1 ("Risk-engine foundation"), Phase 7; DS-RSK-001/002

Purpose: Represent the current computed risk posture (per-trade, daily, weekly, portfolio-level) the Risk Engine evaluates against.

Key Fields: `risk_state_id`, `portfolio_id` (FK), `as_of_timestamp`, `daily_loss_used`, `weekly_loss_used`, `open_position_count`, `sector_concentration` (JSON), `correlated_exposure`

(JSON), drawdown_by_strategy (JSON).

Key Relationships: Read by the Risk Engine stage of the TradeValidationPipeline (DS-ARC-011, DS-ARC-012); one current RiskState per Portfolio, with historical snapshots retained for audit.

Constraints/Invariants: Computed deterministically (DS-RSK-002, now Committed per DS-002 v0.4.0); a stale RiskState (computed before the most recent Order) is not used to gate a new trade — the Risk Engine recomputes rather than trusting a cached snapshot for gating decisions.

Testing: Deterministic recomputation regression test (DS-RSK-002).

DS-DB-012 — BrokerState

Release Classification: Planned (corrected in the DS-005-A03 repair — ROADMAP.md Phase 7 paper-broker/reconciliation architecture is approved Planned work, not Future/Exploratory; live-mode use remains gated behind Phase 13) | **Governing Source:** ARCHITECTURE.md §6, §14; ROADMAP.md Phase 7 (paper), Phase 13 (live); DS-ARC-021 (Broker and Auto-Trader Architecture, Planned)

Purpose: Represent DarkSage's internal view of broker-side account state, for reconciliation (TRADING_RULES.md/SECURITY_RULES.md "Broker Reconciliation").

Key Fields: broker_state_id, portfolio_id (FK), broker_environment, positions_snapshot (JSON), cash_snapshot, open_orders_snapshot (JSON), last_reconciled_at, reconciliation_status (ok/mismatch).

Key Relationships: One-to-one (current) with Portfolio per environment; compared against Position/Order tables during reconciliation.

Constraints/Invariants: A reconciliation_status = mismatch pauses new trading and raises an audit event (SECURITY_RULES.md "Broker Reconciliation"), consistent with the fail-closed principle.

Testing: Not yet applicable — Paper Broker reconciliation is Phase 7; live is Phase 13. Recorded now because the entity shape is already implied by existing rules, not because it is committed.

9. Chart Entities

DS-DB-013 — ChartAnnotation

Release Classification: Planned | **Governing Source:** ARCHITECTURE.md §6, §9; DS-CHT-003

Purpose: Store chart annotations (trendlines, notes, markers), authored either by the user directly or by Sage, per DS-CHT-003.

Key Fields: annotation_id, symbol_id (FK), chart_config_id (nullable), type (trendline/note/marker/fibonacci), geometry (JSON — anchored to price/time, not pixel position), author (user/ai), created_at.

Key Relationships: Many-to-one with SecurityIdentity; independent of price/indicator data (presentation-layer only, per DS-PRD-003 — annotations never affect calculation).

Constraints/Invariants: geometry is anchored to price/time coordinates so it remains correctly positioned across timeframe/rescale changes (DS-CHT-003's edge case).

Testing: Rescale-anchoring regression test (DS-CHT-003).

10. Product and Platform Entities

DS-DB-014 — SecurityIdentity

Release Classification: Committed / MVP | **Governing Source:** DS-DAT-003 (Committed per DS-002 v0.4.0); ARCHITECTURE.md §6

Purpose: The canonical internal identity for a tradable security, decoupled from its display ticker, so a ticker/symbol change doesn't break references.

Key Fields: `symbol_id` (canonical, immutable, e.g., a surrogate key or FIGI-style identifier), `current_ticker`, `instrument_type` (stock/ETF/option/crypto/future), `exchange`, `delisted_at` (nullable).

Key Relationships: Referenced by nearly every other entity in this document (Candle, Quote, Signal, Position, Watchlist, ChartAnnotation) as the stable foreign key.

Constraints/Invariants: `symbol_id` is never reused after a delisting/reuse event, even if `current_ticker` is later reused by a different company (DS-DAT-003's disambiguation edge case).

Testing: Ticker-reuse disambiguation test (DS-DAT-003).

DS-DB-015 — UserProfile

Release Classification: Planned | **Governing Source:** DS-USR-006 (Planned per DS-002 v0.3.0)

Purpose: Minimal local user identity scoping preferences and workspace state, without requiring an external account.

Key Fields: `profile_id`, `display_name`, `created_at`.

Key Relationships: One-to-many with Preferences (DS-DB-016), WorkspaceLayout (DS-DB-017), Watchlist (DS-DB-018).

Constraints/Invariants: No external-account dependency for core local functionality (DS-USR-006).

Testing: Offline/no-account functional test.

DS-DB-016 — Preferences

Release Classification: Planned | **Governing Source:** DS-USR-002, DS-USR-003

Purpose: Persist user-configurable settings (terminology mode, notification settings, etc.).

Key Fields: `profile_id` (FK), `key`, `value` (JSON), `updated_at`.

Key Relationships: Many-to-one with UserProfile.

Constraints/Invariants: A corrupted/unreadable preference falls back to a documented safe default (DS-USR-002), not a crash.

Testing: Corrupted-storage recovery test.

DS-DB-017 — WorkspaceLayout

Release Classification: Planned (baseline shell is Committed per DS-WKS-001; saved multi-layout management remains Planned per DS-WKS-003) | **Governing Source:** DS-WKS-001, DS-WKS-003

Purpose: Persist workspace widget composition, position, and size.

Key Fields: layout_id, profile_id (FK), name, widget_composition (JSON), is_active, created_at.

Key Relationships: Many-to-one with UserProfile.

Constraints/Invariants: A layout referencing a widget type that no longer exists degrades gracefully on load (DS-WKS-003's edge case), not a load failure.

Testing: Missing-widget-type degradation test.

DS-DB-018 — Watchlist / WatchlistItem

Release Classification: Planned | **Governing Source:** DS-MKT-007 (Planned per DS-002 v0.4.0)

Purpose: A named, curated set of symbols a user tracks.

Key Fields: Watchlist: watchlist_id, profile_id (FK), name. WatchlistItem: watchlist_id (FK), symbol_id (FK), added_at.

Key Relationships: Many-to-many between UserProfile (via Watchlist) and SecurityIdentity.

Constraints/Invariants: A WatchlistItem referencing a delisted SecurityIdentity is flagged, not silently removed (DS-MKT-007).

Testing: Delisted-symbol flag test.

11. Security and Operations Entities

DS-DB-019 — IntegrationCredential

Release Classification: Committed / MVP | **Governing Source:** DS-INT-002 (Committed); DS-SEC-001; SECURITY_RULES.md

Purpose: Represent a stored external-integration credential (market-data provider, AI provider, future broker), using secure storage rather than plaintext.

Key Fields: credential_id, integration_type (market_data/ai_provider/broker), provider_name, secure_storage_reference (opaque pointer into OS credential store/encrypted vault — never the raw secret value itself), status (active/disabled/invalid), created_at, last_validated_at.

Key Relationships: Referenced by provider-adapter configuration (DS-ARC-006, DS-ARC-013).

Constraints/Invariants: The table never stores a raw secret value in a plain column — only an opaque reference to secure storage (DS-SEC-001, DS-ARC-015); no query against this table can return a usable secret.

Testing: Log/export scan test confirming absence of secret values (DS-SEC-001).

DS-DB-020 — AuditLogEntry

Release Classification: Committed / MVP | **Governing Source:** DS-OPS-001/002 (Committed)

Purpose: Append-only record of material application events (startup, ingestion failures, Risk Engine determinations, AI outages, crashes, security-sensitive actions).

Key Fields: entry_id, event_category, timestamp, correlated_entity_id (nullable, e.g., a TradeDecision or IntegrationCredential id), detail (JSON, secret-redacted), severity.

Key Relationships: Loosely referenced (by id, not FK constraint) from TradeDecision, RiskState, IntegrationCredential, and other event-producing entities.

Constraints/Invariants: Append-only — no update/delete path in normal operation; detail is redacted of secrets before write (DS-OPS-001, DS-SEC-001).

Testing: Log-content secret-redaction audit; audit-reconstruction test (DS-OPS-002).

12. Scanner Entities

DS-DB-021 — ScanConfiguration

Release Classification: Committed / MVP | **Governing Source:** DS-SCN-002 (Committed per DS-002 v0.4.0)

Purpose: A saved, reusable scan definition (universe, filter criteria, ranking configuration).

Key Fields: scan_id, profile_id (nullable FK — corrected in the DS-005-A03 repair; null resolves to the implicit single local profile when the Planned DS-USR-006 multi-profile feature is not yet enabled, so this Committed entity does not require a Planned feature to be independently satisfiable), name, universe_definition (JSON — e.g., watchlist reference, index membership, explicit list), filter_criteria (JSON), ranking_config (JSON), created_at.

Key Relationships: One-to-many with ScanResult (DS-DB-022).

Constraints/Invariants: universe_definition is evaluated against current membership at run time, with material changes disclosed (DS-SCN-002's edge case). A null profile_id is valid and represents scan configurations owned by the implicit default local profile.

Testing: Empty-universe test; scan save/execute regression test.

DS-DB-022 — ScanResult

Release Classification: Planned | **Governing Source:** DS-SCN-005 (Planned)

Purpose: Historical record of a scan execution's output, for trend review.

Key Fields: result_id, scan_id (FK), executed_at, matched_symbols (JSON, with per-symbol rank/score), universe_snapshot_size.

Key Relationships: Many-to-one with ScanConfiguration.

Constraints/Invariants: Immutable once written (historical results remain historical, DS-SCN-005); subject to a documented retention policy to bound storage growth.

Testing: Scan-history retrieval test confirming historical immutability.

13. Alerts and Notifications

Added in the DS-005-A02 repair to close a gap: DS-ALT-001/002 (Planned) require configured alerts and retrievable notification history/recovery, but no prior DS-005 entity supported them (Signal referenced "Alert" without a defined target).

DS-DB-026 — Alert

Release Classification: Planned | **Governing Source:** DS-ALT-001 (User-Configured Alerts, Planned)

Purpose: A user-configured, persistent alert condition (market data, watchlist event, or risk-limit condition per DS-ALT-001).

Key Fields: alert_id (primary identifier), profile_id (nullable FK → UserProfile, DS-DB-015 — same Phase-1-independence pattern as DS-DB-021 ScanConfiguration), name, condition_definition (JSON — e.g., symbol/indicator/threshold or risk-limit reference), status (active/disabled), created_at, last_triggered_at (nullable).

Key Relationships: One-to-many with Notification (DS-DB-027); optionally references a SecurityIdentity (DS-DB-014) or RiskState (DS-DB-011) condition target within condition_definition.

Constraints/Invariants: An Alert firing never creates an Order or any execution-path record — it produces only a Notification (DS-PRD-007's notification-only boundary; DS-EXE-001's pipeline-is-the-only-path-to-an-order boundary remains untouched by this entity).

Testing: Not yet applicable at Planned classification; deferred to Phase-appropriate authoring alongside DS-ALT-001's own implementation.

DS-DB-027 — Notification

Release Classification: Planned | **Governing Source:** DS-ALT-001 (fired-alert record), DS-ALT-002 (Notification Delivery, Planned)

Purpose: The durable record of a fired alert or system event, sufficient to survive an application restart so a notification missed while closed/backgrounded is visible on next open (DS-ALT-002's edge case).

Key Fields: notification_id (primary identifier), alert_id (nullable FK → Alert — nullable because a Notification may also originate from a non-Alert system event, e.g., a Risk Engine warning per DS-RSK-003 once that becomes Committed), triggered_at, condition_snapshot (JSON — the specific condition/value that triggered it, per DS-ALT-001's "timestamp, condition, triggering value"), delivered_at (nullable), read_at (nullable), channel (in_app / external — both Planned: DS-ALT-002 governs in-app delivery, DS-ALT-003 governs external-channel delivery).

Key Relationships: Many-to-one with Alert (when alert-originated); loosely referenced from other event-producing entities (RiskState, TradeDecision) the same way AuditLogEntry is (DS-DB-020), by id rather than a hard FK constraint.

Constraints/Invariants: Persisted immediately on trigger (not only on delivery) so that an application closed at fire time still shows the notification on next open, per DS-ALT-002's restart-recovery requirement; a Notification is never itself a trade/order record (same DS-PRD-007 boundary as Alert).

Testing: Not yet applicable at Planned classification; deferred to Phase-appropriate authoring. Restart-recovery test (notification visible after relaunch) is the key acceptance test once implemented.

14. Cross-Cutting Data Integrity Requirements

DS-DB-023 — Append-Only Audit and Transaction Tables

Priority: Critical | **Release Classification:** Committed / MVP | **Governing Source:** DS-PRD-004; DS-OPS-002; AGENTS.md "Database Changes"

Description: Tables recording historical fact (AuditLogEntry, BacktestResult, Transaction — DS-DB-025) shall be append-only in normal operation; corrections use an explicit, logged reversing/adjusting entry, never an in-place edit or delete.

Acceptance Criteria: No application code path issues an UPDATE or DELETE against these tables outside a documented, audited correction procedure.

Testing: Static analysis / code-review check for UPDATE/DELETE statements against append-only tables.

DS-DB-024 — Migration Discipline

Priority: High | **Release Classification:** Committed / MVP | **Governing Source:** AGENTS.md "Database Changes"

Description: Once migrations are introduced, all schema changes shall use versioned migrations; destructive schema changes shall require impact review and a documented migration/backup path.

Acceptance Criteria: No schema change ships without a corresponding migration script and rollback/backup note once the migration system exists.

Testing: Migration dry-run/rollback test (once the migration system exists; not yet applicable pre-Phase-1 tooling selection).

15. Non-Goals

DS-005 does not: select a database engine beyond SQLite-first (DS-ARC-016 already governs this); define exact SQL column types/indexes (implementation detail); or define API serialization shapes (DS-006).

16. Dependencies

- DS-001, DS-002, DS-004
- ARCHITECTURE.md §6, §20; PROJECT_SPEC.md §7, §16, §17; TRADING_RULES.md; SECURITY_RULES.md; AGENTS.md

17. Risks and Constraints

- **Sequencing risk:** authored before DS-006 (API contracts); entity `_snapshot/JSON-blob` fields are provisional shape, refined once API serialization needs are known.
- **Classification discipline:** the DS-005-A03/A05 repair applied the same "Phase-1 core vs. later-phase extension" field-splitting pattern used for DS-ARC-005/DS-ARC-004 (DS-004) and DS-AI/DS-SGE (DS-002/DS-003) to `StrategyProfile` and `TradeDecision`, so a Committed base model never requires a Planned capability to be valid.

18. Verification Approach

Entity-level Testing per DS-DB-NNN item. Document-level verification (unique-ID check, cross-reference consistency, no contradiction with DS-002/DS-003/DS-004) recorded in `.ai-workflow/HANDOFF.md`.

19. References

- `ARCHITECTURE.md`, `PROJECT_SPEC.md`, `TRADING_RULES.md`, `SECURITY_RULES.md`, `AGENTS.md`
- `docs/codex/Volume-02-Product/DS-002-SRS.md` and `requirements/*.md`
- `docs/codex/Volume-04-Architecture/DS-004-Technical-Architecture.md`

Appendix A — Open Questions

1. **RESOLVED in the DS-005-A01 repair.** Transaction entity (DS-DB-025) added; Position and DS-DB-023 updated to reference it. Retained here for revision-history traceability, not as an open item.
2. **JSON-blob field granularity** — several entities use JSON columns (e.g., `pipeline_stage_results`, `configuration`) as a placeholder for structured sub-schemas better suited to normalized tables. Acceptable for a first draft; a later revision should normalize where query patterns justify it (DS-006/DS-004 input).
3. **RESOLVED in the DS-005-A02 repair.** Alert (DS-DB-026) and Notification (DS-DB-027) entities added, supporting DS-ALT-001/002 and preserving the DS-PRD-007 notification-only boundary. Retained here for revision-history traceability, not as an open item.
4. **Nullable `profile_id` pattern (DS-005-A03 repair)** — `ScanConfiguration` and `Alert` both use a nullable `profile_id` resolving to an implicit default local profile so they remain independently satisfiable without the Planned multi-profile `UserProfile` feature (DS-USR-006). This is a routine implementation pattern, not a governance question, but is noted here since it's a repeated design choice worth keeping consistent if more Committed entities gain a similar dependency in the future.